Competitive Programming

Section 45: Flows and Matchings

Competitive Programming Course

2026-10-26

Table of contents

1 Start with five units to deliver 3

2 Reversible choices let us repair a routing 6

3 Dinic: organize the searches into phases 12

4 Minimum cut: its capacity equals the maximum flow 30

5 Dinic: complete reusable implementations 32

6 Matching: a concrete reassignment 38

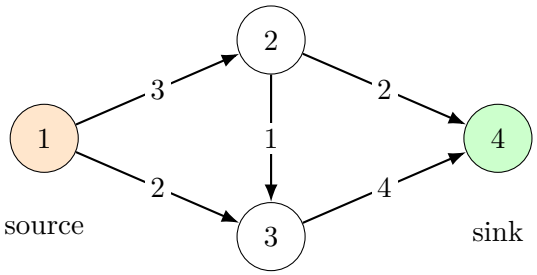
7 Matching: complete reusable implementations 47

8 Explore and test the algorithms 50

9 Exercises 52

1 Start with five units to deliver

Imagine that vertex `1` has material to send to vertex `4`. An arrow's capacity is the most material it can carry. Vertices `2` and `3` may forward material, but cannot create or keep it. How much can reach `4` altogether?

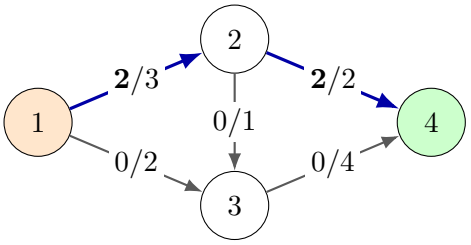


Start with nothing on any arrow. We will build a delivery by choosing one complete route at a time. For now, follow the amounts in the drawings; we will study how an algorithm finds its routes later.

In the next three drawings, each label is **amount already sent / capacity**, after the step. For example, `2/3` means that the arrow carries `2` units and can still carry `1` more. Blue arrows show the route used in that step; the other arrows keep their previous amounts.

1.1 First route: send two units through vertex 2

Choose `1 -> 2 -> 4`. The first arrow can carry `3` units, but the second can carry only `2`. Send **2 units along both arrows**: vertex 2 receives 2 and immediately forwards 2 to vertex 4. Sending 3 would overload `2 -> 4`.



Added on this route: 2

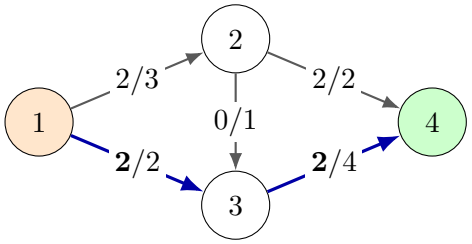
Total delivered: 2

Vertex 2: 2 arrives, 2 leaves.

Edge $2 \to 4$ is now full.
Edge $1 \to 2$ still has room for 1.

1.2 Second route: send two more units through vertex 3

Keep the first delivery in place. On `1 -> 3 -> 4`, neither arrow has been used yet: they have room for 2 and 4 units respectively. Send **2 more units along both arrows**. The receiver now gets 2 through vertex 2 and 2 through vertex 3, for a total of 4.



Added on this route: 2

Total delivered: 4

Vertex 3: 2 arrives, 2 leaves.

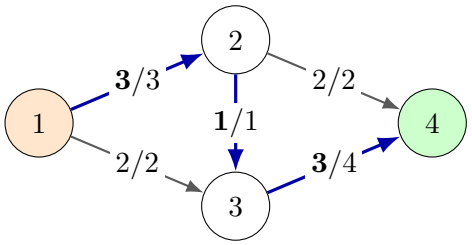
Edge $1 \to 3$ is now full.
Edge $3 \to 4$ still has room for 2.

1.3 Third route: use the remaining room between the branches

Four is not the limit. Although `2 -> 4` is full, we can still go from vertex 2 through vertex 3 to reach vertex 4. Look at the previous drawing:

- `1 -> 2` carries 2 of its capacity 3: room for **1** more.
- `2 -> 3` carries 0 of its capacity 1: room for **1** more.
- `3 -> 4` carries 2 of its capacity 4: room for **2** more.

So route `1 -> 2 -> 3 -> 4` can carry **1 additional unit**. Add it on each of these three arrows, leaving every other amount unchanged:



Added on this route: 1

Total delivered: 5

Vertex 2: 3 arrives, $2 + 1$ leaves.

Vertex 3: $2 + 1$ arrives, 3 leaves.

Both junctions still balance.

The amount an entire route can add is limited by its smallest remaining room. This amount is called its **bottleneck**: here it was 2, then 2, then 1. Notice that the final labels show the accumulated amounts, not just the last unit added: `1 -> 2` now carries 3, and `3 -> 4` now carries 3.

1.4 Why five is the maximum

The receiver gets `2 + 3 = 5`. No larger delivery is possible because the only arrows leaving vertex 1 have capacities 3 and 2, and both are full. Even though `3 -> 4` still has room for one more unit, no additional unit can leave the source to reach it.

We have both a feasible routing of 5 units and an upper bound of 5. This is the basic proof pattern we will reuse for maximum flow.

1.5 Vocabulary for the example

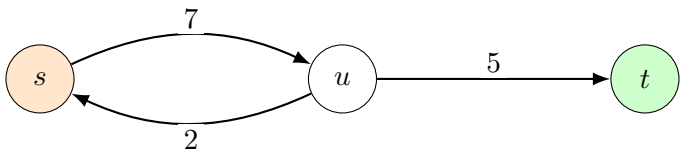
A **flow network** is a directed graph, a nonnegative capacity for each input edge, and distinct vertices `s` (source) and `t` (sink). Each input edge has its own flow, even when several arrows have the same endpoints.

A feasible flow satisfies two rules:

```
0 <= flow[edge] <= capacity[edge]
for each vertex except s and t: total incoming flow = total outgoing flow
```

The **flow value** measures how much the source supplies to the sink. In our final drawing, it is **5**: `3 + 2` leaves vertex 1, and `2 + 3` reaches vertex 4. Do not add the amounts on every edge of the graph: that would count the same unit again each time it passes through another edge.

A return to the source must be subtracted. Some networks have actual input edges returning to the source. In the drawing below, the source sends 7 units to `u`, but `u` sends 2 back and forwards only 5 to the sink. The value is therefore **`7 - 2` = 5**, not 7. Labels here are the amounts carried; capacities are omitted. Both opposite arrows are real input edges.



supplies $7 - 2 = 5$ receives 7, forwards $2 + 5$ receives 5

This gives the general definition:

```
flow value = total leaving s - total returning to s
           = total entering t - total leaving t
```

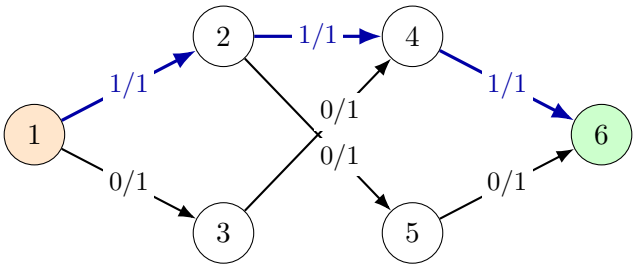
The two calculations agree because every intermediate vertex forwards exactly what it receives: it creates nothing and keeps nothing. The amount supplied by the source must therefore equal the amount absorbed by the sink. If no input edge enters `s` or leaves `t`, as in our opening network, the subtracted amounts are simply zero.

The optimization goal. We want the largest flow value among all assignments satisfying the capacity and conservation rules. We can start with zero on every edge: no capacity is exceeded, and each junction receives and forwards zero. We will then improve this assignment while keeping both rules true.

A maximum flow need not fill every edge. In the opening example, `3 -> 4` carries only 3 of its capacity 4, yet the value 5 is maximum because no more can leave the source. The algorithm maintains the edge amounts; an exercise asking only for the maximum value needs just the number `5` as output.

2 Reversible choices let us repair a routing

In the opening example, we never changed an amount already assigned to an edge. We now examine a strategy that repeatedly chooses a route with room left and sends flow along it. An early route can be perfectly feasible yet need revising later. Consider this network, where every capacity is `1`.

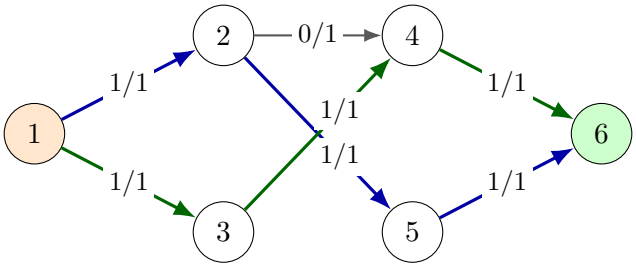


After choosing route $1 \to 2 \to 4 \to 6$.
Labels are flow/capacity.

The route `1 -> 2 -> 4 -> 6` carries one unit and respects every rule. But keeping it unchanged blocks further delivery: the remaining branch `1 -> 3 -> 4` cannot reach the sink because `4 -> 6` is full, and `1 -> 2` is also full. A search restricted to unused forward capacity stops at value 1.

2.1 A different first choice would succeed

We could instead have chosen `1 -> 2 -> 5 -> 6` first, then `1 -> 3 -> 4 -> 6`. These two routes fit together and deliver 2 units:



Blue: $1 \to 2 \to 5 \to 6$. Green: $1 \to 3 \to 4 \to 6$.
Total delivered: 2. Labels are flow/capacity.

So cancellation is not required in every execution. On this graph, a good first choice avoids it. The issue is how to choose a first route before we know the final arrangement. All three initial routes have three edges and a bottleneck of 1; choosing a shortest route or a route with the largest bottleneck does not distinguish the unfortunate choice from the other two.

2.2 Let an early choice remain provisional

We want a method that can take a feasible route without first proving that it belongs to some optimal final routing. To achieve that, earlier amounts must remain adjustable. Otherwise, after the first drawing, the method would incorrectly conclude that no improvement is possible.

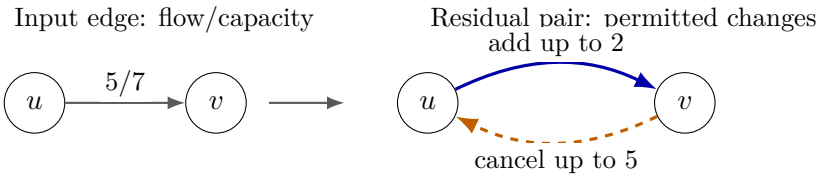
We could abandon earlier routes and try alternatives. The residual graph provides a systematic way to express just the adjustments needed. In this example, keep the unit on `1 -> 2` and `4 -> 6`, cancel its use of `2 -> 4`, and add flow on the other branch edges. That produces exactly the two-unit routing in the second drawing. The next subsection explains how one search can describe all those changes together.

2.3 Residual capacity means permission to change flow

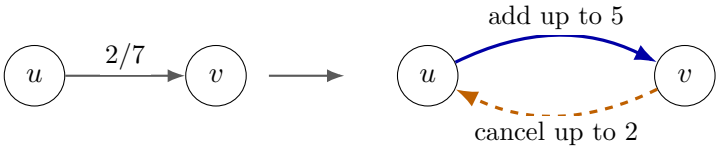
For an input edge `u -> v` with capacity `c` and current flow `f`, maintain **two residual edges**:

The forward residual arrow `u -> v` has capacity `c - f`: room to add more flow. The backward residual arrow `v -> u` has capacity `f`: room to cancel flow already assigned to this input edge.

For capacity `7` and current flow `5`, these permissions are `2` and `5`. The two rows below show what changes when we use the backward permission to cancel `3` units:



Cancel 3 units of the input edge's flow. Its capacity remains 7.



Blue increases input flow. Dashed orange decreases input flow.

Read the bottom row: the input flow is now `5 - 3 = 2`, so there is room to add `7 - 2 = 5` units or to cancel the remaining `2`. These are amounts we may change, not two additional flows already travelling through the graph.

A backward residual arrow is permission to cancel an earlier decision. It need not be an arrow in the input. If the input also has `v -> u`, that input edge gets its own pair; never combine it with this cancellation edge.

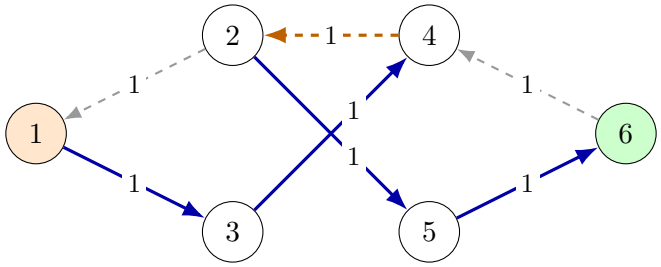
An **augmenting path** is a path from `s` to `t` using positive residual capacities. Its bottleneck is the minimum residual capacity on the path. Pushing that amount changes each input flow in the direction indicated by the corresponding residual edge.

2.4 Trace the cancellation

For the six-vertex example, use residual path

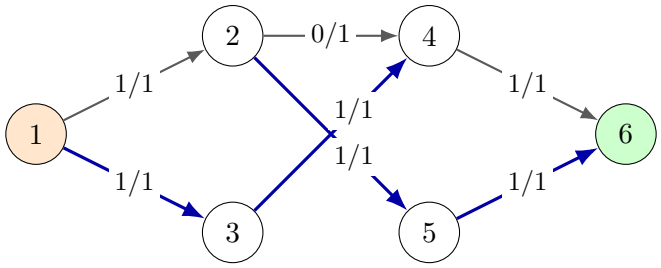
```
1 -> 3 -> 4 -> 2 -> 5 -> 6
```

The step `4 -> 2` cancels the old unit on input edge `2 -> 4`. All five residual capacities on the path are `1`, so the bottleneck is `1`.



Positive residual capacities only. The chosen path is $1 \to 3 \to 4 \to 2 \to 5 \to 6$.
Its orange step $4 \to 2$ cancels the old flow on input edge $2 \to 4$.
Gray dashed arrows are other available cancellations, unused by this path.

Now apply the path as one revision: add one unit on its four blue input edges, and remove the unit on `2 -> 4`. The resulting input flow is:



Final input flows, labelled flow/capacity. Total delivered: 2.
Edge $2 \to 4$ is unused; $1 \to 2$ and $4 \to 6$ keep their original unit.

Edges `1 -> 2` and `4 -> 6` keep their old unit. Read the final input flows: they form the two routes promised above. We have not sent physical material backward along an input road; we have replaced one feasible routing by another.

Why is an augmentation safe? The bottleneck protects every capacity bound. At an internal path vertex, the two incident changes balance, whether a step adds or cancels input flow. The source gains the bottleneck in net outgoing flow and the sink gains the same amount in net incoming flow.

The minimum-cut proof later in this lesson will establish a stronger guarantee: **whenever the current feasible flow is not maximum, its residual graph has an augmenting path**. Thus the method can keep improving from any feasible routing, including one obtained from an unfortunate early choice. Reverse edges are what preserve this guarantee when earlier amounts need to move.

2.5 Store paired edge IDs

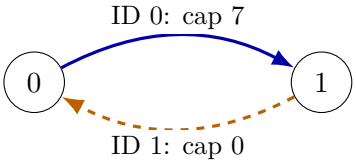
The code needs to store the residual arrows we have just drawn. Whenever it uses one arrow, it must also update that arrow's cancellation partner. Give each residual arrow a unique **edge ID**, its index in a global array. One input edge creates two stored entries.

2.5.1 Create the pair before sending any flow

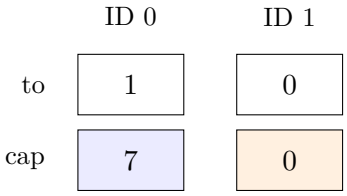
Start with vertices `0` and `1` and no edges. Adding the input edge `0 -> 1` with capacity `7` creates:

- **ID 0**, from vertex 0 to vertex 1, with residual capacity 7: we may send up to 7 units on the input edge.
- **ID 1**, from vertex 1 to vertex 0, with residual capacity 0: we have sent nothing yet, so there is nothing to cancel.

These are two representations of what we may change on the **same input edge**. Adding the reverse entry does not add a second input link.



Circles: vertex numbers.
Arrows: residual edge IDs.



Each column stores one entry.
The reverse starts at zero.

Each stored entry has a destination `to` and a remaining permission `cap`. Thus `to[0] = 1`, `cap[0] = 7`, `to[1] = 0`, and `cap[1] = 0` initially. The vertex numbers and the edge IDs are separate numbering systems.

For each vertex, `adj` lists the IDs of residual arrows leaving it:

```
adj[0] = [0]    from vertex 0, examine residual entry 0
adj[1] = [1]    from vertex 1, examine residual entry 1
```

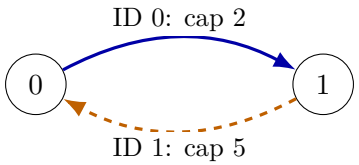
Store even the zero-capacity reverse entry. It will become usable after flow is sent. A search follows an entry only when its current capacity is positive.

In the Python implementation, these fields are `self.to[id]` and `self.cap[id]`. C++ stores them together as `edges[id].to` and `edges[id].cap`. Below, `cap[id]` is shorthand for either language's residual capacity field.

2.5.2 Every push changes both entries

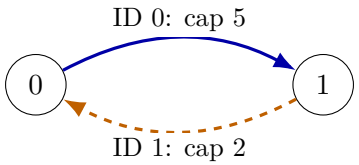
Send 5 units using ID 0. Its permission falls from 7 to 2, and the permission to cancel that flow, ID 1, rises from 0 to 5. Later, cancelling 3 units means using ID 1: its permission falls from 5 to 2, while ID 0 regains 3 units of forward room.

First send 5 units on ID 0. Input flow becomes 5.




```
cap[0] = 7 - 5 = 2
cap[1] = 0 + 5 = 5
```

Then cancel 3 units by using ID 1. Input flow becomes 2.




```
cap[1] = 5 - 3 = 2
cap[0] = 2 + 3 = 5
```

IDs and destinations never change. Only the two residual capacities change.

This is why the same two assignments work in either direction:

```
cap[chosen ID]  -= amount
cap[partner ID] += amount
```

After the second row, the input edge carries `5 - 3 = 2` units. The two residual capacities are 5 and 2, whose sum remains the original capacity 7. The reverse member's capacity is exactly this input edge's current flow.

2.5.3 Find the partner without searching

Create each pair consecutively, starting at ID zero:

```
first input edge:   IDs 0 and 1
second input edge:  IDs 2 and 3
third input edge:   IDs 4 and 5
```

An even ID's partner is the next ID; an odd ID's partner is the previous ID. In both C++ and Python, bitwise XOR with 1 performs exactly that exchange:

```
0 ^ 1 = 1       1 ^ 1 = 0
2 ^ 1 = 3       3 ^ 1 = 2
4 ^ 1 = 5       5 ^ 1 = 4
```

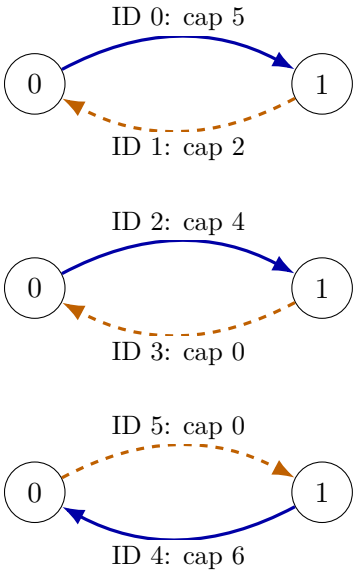
Here `^` means **bitwise XOR**, not exponentiation. It toggles the last binary bit, exchanging the two consecutive members of a pair. The update therefore becomes:

```
cap[edge]     -= delta
cap[edge ^ 1] += delta
```

For the cancellation above, `edge = 1`, so `edge ^ 1 = 0`: decrease the backward permission and increase the forward permission. No special backward case is needed. `add_edge` returns the even ID so callers can identify the original input edge later.

2.5.4 Keep different input edges in different pairs

Continue the same example, where the first edge now carries 2 units. Add another input edge `0 -> 1` with capacity 4, then an input edge `1 -> 0` with capacity 6. They create two new pairs; they do not reuse the old pair.



Input edge A: $0 \to 1$.
Capacity 7; current flow 2.

Input edge B: $0 \to 1$.
Capacity 4; current flow 0.

Input edge C: $1 \to 0$.
Capacity 6; current flow 0.

Each row shows a separate pair between the same two vertices.
ID 1 cancels edge A's flow; ID 4 adds flow on edge C.

In particular, IDs 1 and 4 both point from vertex 1 to vertex 0, but they have different meanings. Using ID 1 reduces flow on input edge A; using ID 4 increases flow on input edge C. Their partner IDs are 0 and 5 respectively. Knowing only the destination vertex would not tell us which partner to update.

The outgoing lists now contain:

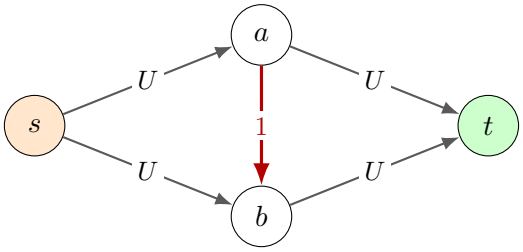
```
adj[0] = [0, 2, 5]
adj[1] = [1, 3, 4]
```

An outgoing list contains **edge IDs**, not destination vertices. Its local positions are different from those IDs: position 1 of `adj[0]` contains edge ID 2, and `to[2]` tells us that this edge leads to vertex 1. Later, Dinic's current pointer will remember a position in such a list.

3 Dinic: organize the searches into phases

Residual paths let us repair earlier choices, but correctness alone does not guarantee speed. Even with integer capacities, a poor choice of paths can make progress depend on the amount of flow rather than the graph size.

Consider this network. All four outer edges have capacity `U`; the central edge has capacity 1. Take `U = 1 000 000 000`:



Input capacities.

Maximum flow: $2U$. Send U on each outer branch.

Arbitrary paths can alternate between using and cancelling the central edge. Starting from zero, choose:

1. `s -> a -> b -> t`: send 1, limited by `a -> b`.
2. `s -> b -> a -> t`: send 1, using the residual arrow `b -> a` to cancel the unit on `a -> b`.

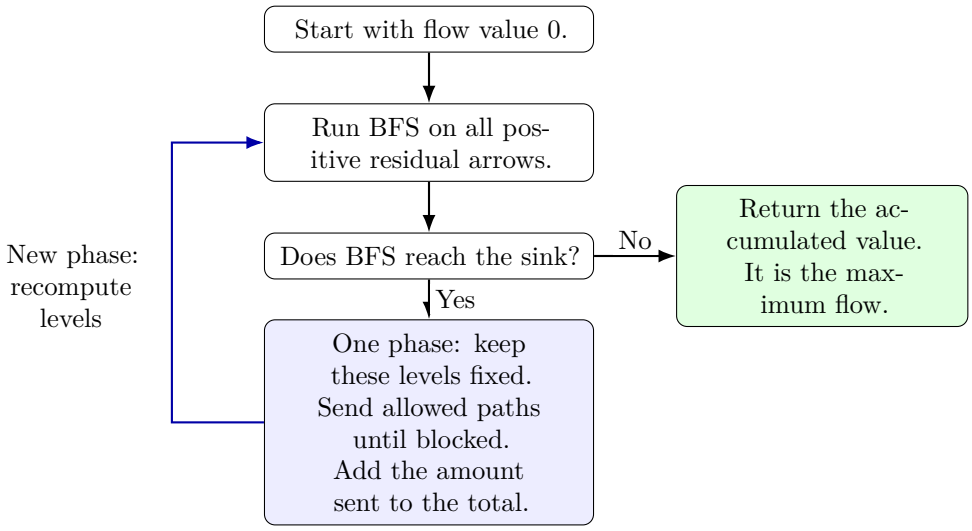
After these two augmentations, each outer input edge carries 1, while the central edge again carries zero. The total value is 2. Repeat the same pair of choices: each pair adds only 2, and restores the central edge to zero flow. After `k` pairs, every outer edge carries `k` and the value is `2k`. Both choices remain possible until `k = U`.

This legal execution needs **`2U` augmentations to obtain value `2U`**: two billion augmentations on a graph with only four vertices and five edges. The total always increases, so the algorithm terminates, but far too slowly.

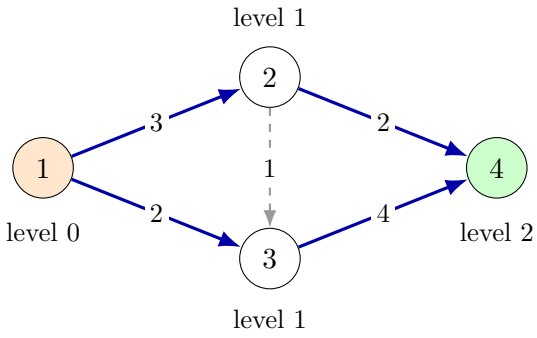
Dinic uses shortest-path levels to avoid this execution. The first BFS assigns `level[s] = 0`, `level[a] = level[b] = 1`, and `level[t] = 2`. During a phase, flow searches may follow only arrows that advance exactly one level. The central arrow `a -> b` stays at level 1, so it is excluded. Dinic sends `U` on `s -> a -> t` and `U` on `s -> b -> t`: **two augmentations in one phase** reach the maximum.

This example exposes the danger of arbitrary paths; other algorithms, such as Edmonds-Karp, also avoid it by choosing shortest residual paths. Dinic goes further: it works through the current shortest-path levels until no allowed source-to-sink path remains, then recomputes the levels. We will show that this organization bounds the number of phases by `O(V)`, independently of the capacity values.

Dinic has two nested loops. The **outer loop** runs BFS on the full residual graph. If the sink is reachable, the **inner loop** sends flow using just the current levels. When that inner loop is blocked, the outer loop runs a fresh BFS and may discover longer routes. The algorithm returns the maximum value only when a full BFS cannot reach the sink.



3.1 Build a level graph with BFS



Count arrows from vertex 1.

Vertices 2 and 3 need one step.
Vertex 4 needs two steps.

Initial residual capacities; no flow has been sent yet.

`level` is an array with one entry per vertex. **`level[v]` is the smallest number of usable residual arrows needed to reach vertex `v` from the source.** Usable means positive residual capacity. Each arrow counts as one step, regardless of its capacity; the level is a distance, not an amount of flow.

In the opening network, before sending flow, the source is vertex 1:

```
level[1] = 0    already at the source: no arrow needed
level[2] = 1    follow 1 -> 2
level[3] = 1    follow 1 -> 3
level[4] = 2    follow 1 -> 2 -> 4, or 1 -> 3 -> 4
```

"Set `level[4]` to 2" means storing `2` in vertex 4's array entry. The longer route `1 -> 2 -> 3` does not change `level[3]`: the direct route needs only one step.

BFS computes these distances one layer at a time:

1. Initialize levels to `-1` (not reached), set `level[s] = 0`, and enqueue `s`.
2. Pop a vertex `u`. Inspect all its positive-capacity residual arrows, including cancellation arrows.
3. For each destination `v` still at `-1`, assign `level[v] = level[u] + 1` and enqueue it. Keep an already assigned level: the first discovery is shortest.
4. Repeat until the queue is empty. Vertices still at `-1` are unreachable.

If `level[t] == -1`, no augmenting path exists and the algorithm stops. Otherwise, this phase's flow searches use only arrows advancing one layer, `level[v] == level[u] + 1`. These arrows form the **level graph**. Since `v = to[edge]`, the code condition is:

```
cap[edge] > 0 and level[to[edge]] == level[u] + 1
```

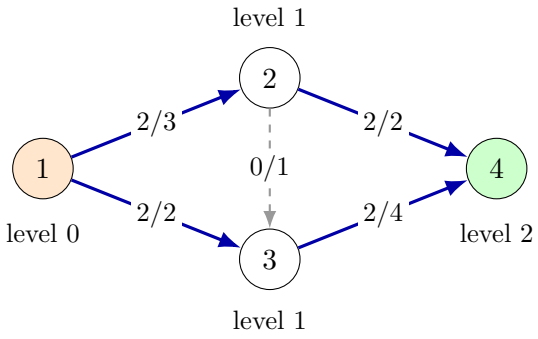
In the drawing above, blue arrows advance one layer. Vertices 2 and 3 both have level 1, so their dashed connecting arrow is excluded from the level graph. Arrow labels are residual capacities; vertex "level" labels are distances. Levels strictly increase along retained arrows, preventing cycles.

3.2 One phase: send a blocking flow

We are now inside **one iteration of Dinic's outer loop**. Keep the current BFS levels fixed, repeatedly find an allowed source-to-sink path, and send its bottleneck. Eventually no path remains that advances through these fixed levels. The extra flow sent during this phase is called a **blocking flow**: every source-to-sink path in the phase's initial level graph now has an edge with zero residual capacity.

This ends the phase, not the algorithm. Keep the flow already sent and run BFS again on the full updated residual graph, including arrows excluded during the phase that just ended. That BFS can reveal a longer route and start another phase. Only when this full BFS cannot reach the sink is the accumulated flow maximum.

For the opening network, first send 2 along `1 -> 2 -> 4`, then 2 along `1 -> 3 -> 4`. In these levels, `2 -> 3` is forbidden, so the phase stops at value 4:

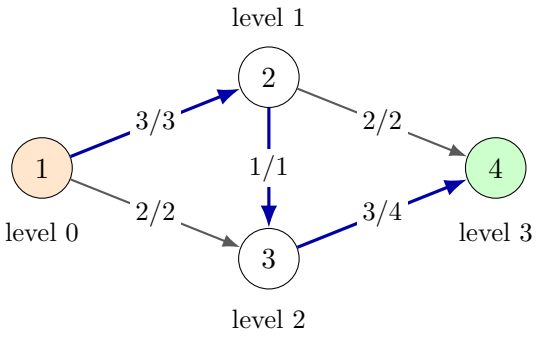


Phase 1 adds $2 + 2 = 4$.

Both two-edge routes are blocked.
Edge $2 \to 3$ is still excluded by the levels.

Flow/capacity after phase 1. Levels are fixed for the whole phase.

The remaining forward capacities along `1 -> 2 -> 3 -> 4` are `1,1,2`. A fresh BFS gives these vertices levels `0,1,2,3`. The previously skipped edge now advances one level, so the next phase sends one more unit:

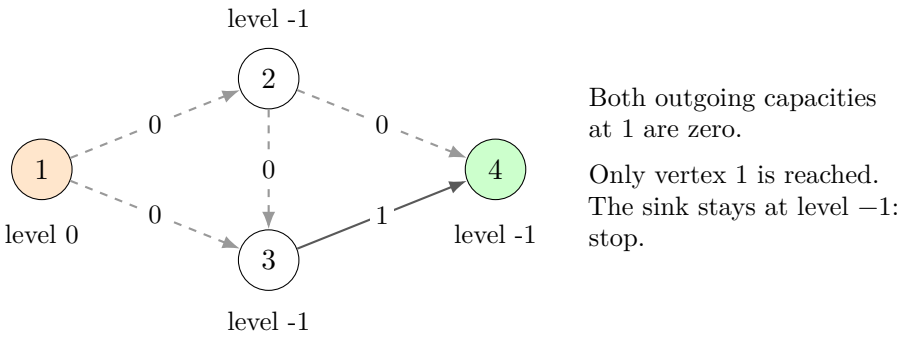


A fresh BFS makes $2 \to 3$ usable.

Phase 2 sends 1 along $1 \to 2 \to 3 \to 4$.
Total flow is now 5.

Flow/capacity after phase 2. The new levels allow a longer path.

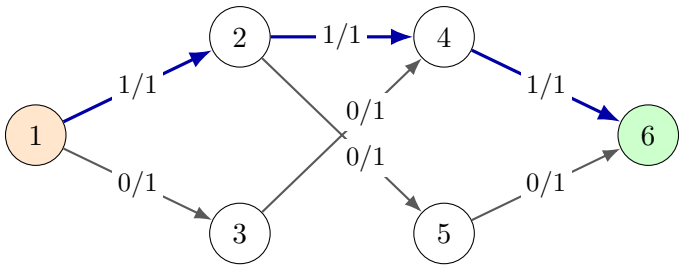
Rebuild BFS once more. It cannot leave the source:



Forward residual capacities after value 5. Dashed zero-capacity arrows are unusable. Positive reverse residual arrows are omitted; none can be reached from 1.

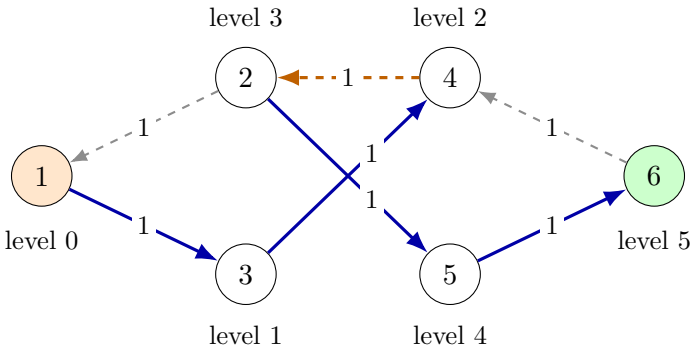
The first phase was blocking but not globally maximum. Rebuilding levels revealed the extra route; the final failed BFS certifies that none remains.

Return to the six-vertex cancellation example. The complete network is repeated here so that the next BFS can be read without returning to chapter 2:



A. The entire input graph after phase 1, labelled flow/capacity.
The blue route $1 \to 2 \to 4 \to 6$ carries one unit. Total flow: 1.

The saturated input arrows `1 -> 2`, `2 -> 4`, and `4 -> 6` now permit cancellation in the opposite direction. Run BFS on all positive residual arrows; it gives the levels shown beside the vertices below:

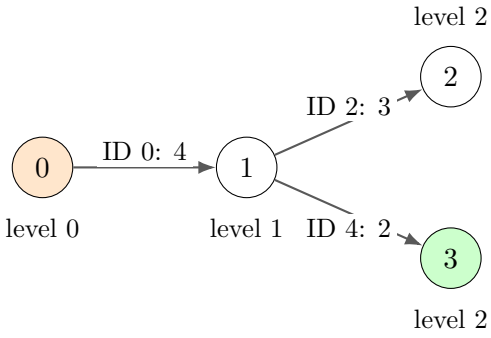


B. All positive residual arrows after phase 1; labels are residual capacities.
Blue and orange form $1 \to 3 \to 4 \to 2 \to 5 \to 6$.
Gray arrows are available cancellations that do not advance one BFS level.

The original input edge is `2 -> 4`. Its orange cancellation arrow is `4 -> 2`: backward relative to that input edge, but from level 2 to level 3 in this new BFS. It therefore satisfies `level[to] == level[u] + 1`, just like the blue arrows. Gray arrows `2 -> 1` and `6 -> 4` go to smaller levels, so this phase excludes them. The highlighted path adds one unit, raising the total from 1 to 2.

3.3 Current pointers and the explicit search stack

Use this small network throughout the subsection. Vertices are 0-based, as in the implementation. The BFS levels are already computed and remain fixed:



Source: 0. Sink: 3.

Vertex 2 is a dead end. The other branch reaches the sink.

Arrow labels are residual edge ID: capacity. Zero-capacity reverse entries are omitted.

3.3.1 First understand the search, without the arrays

We need a complete route from source 0 to sink 3. Start by trying `0 -> 1`, then `1 -> 2`. Vertex 2 has no allowed continuation, so this attempt fails. Return to vertex 1 and try its other branch, `1 -> 3`. That reaches the sink.

Exploring an arrow does not send flow yet. We change capacities only after finding a complete source-to-sink route. Returning from the dead end abandons a search attempt; there is no flow on that branch to cancel.

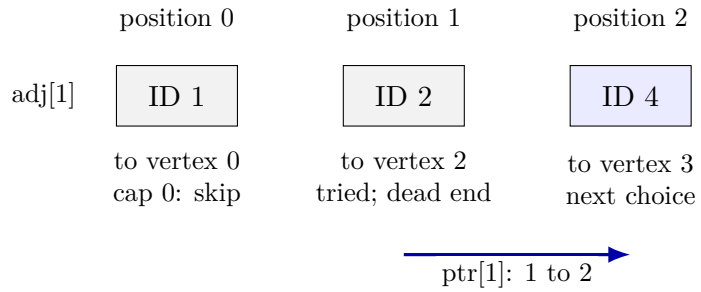
3.3.2 Remember which outgoing entry to try next

The residual pairs were inserted in this order:

```
input 0 -> 1: forward ID 0, reverse ID 1
input 1 -> 2: forward ID 2, reverse ID 3
input 1 -> 3: forward ID 4, reverse ID 5
```

Thus vertex 1's outgoing list is `adj[1] = [1, 2, 4]`. The variable `ptr[1]` is a bookmark: **the position in this list where its search should resume**.

Initially `ptr[1] = 0`, selecting ID 1. This reverse entry has capacity zero, so skip it. Position 1 selects ID 2, which leads to the dead end. Once that attempt fails, advance to position 2, selecting ID 4:



After the dead end, ptr[1] = 2 selects edge ID 4, not edge ID 2.

In general, `edge = adj[u][ptr[u]]` reads the selected edge ID, then `v = to[edge]` reads its destination. These are three separate quantities: list position, edge ID, and destination vertex.

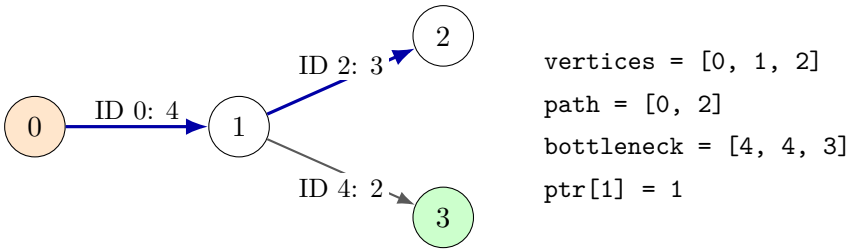
3.3.3 Store the route currently being explored

The iterative search keeps three lists, used as stacks: append to extend the route; remove the last entry to return one step.

- `vertices` lists the visited vertices on the current candidate route. Its last entry tells us where the search is now.
- `path` lists the residual edge IDs joining those vertices. We need the exact IDs to update their capacities and their paired reverse entries.
- `bottleneck` records the minimum capacity along each prefix of that route. Keeping every prefix value lets us recover the previous minimum when we remove the last step.

Start with `vertices = [0]`, `path = []`, and `bottleneck = [4]`. The initial 4 is a safe upper bound: only 4 units can leave the source. After edge ID 0, the minimum is still 4; after edge ID 2, it becomes `min(4,3) = 3`:

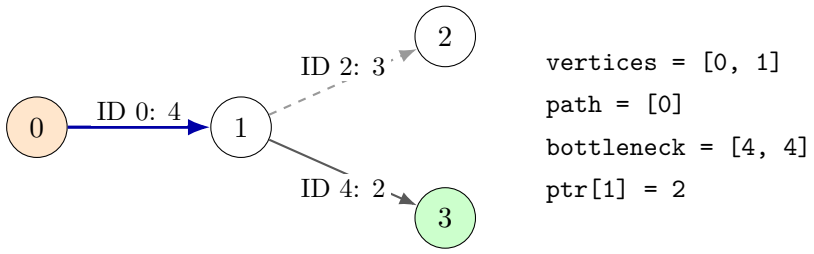
A. Search reaches vertex 2, but no continuation reaches the sink.



The path is only a candidate. No flow has been sent; all capacities are unchanged.

Return to vertex 1: remove vertex 2, the last edge ID 2, and its prefix bottleneck 3. Advance `ptr[1]` from position 1 to position 2:

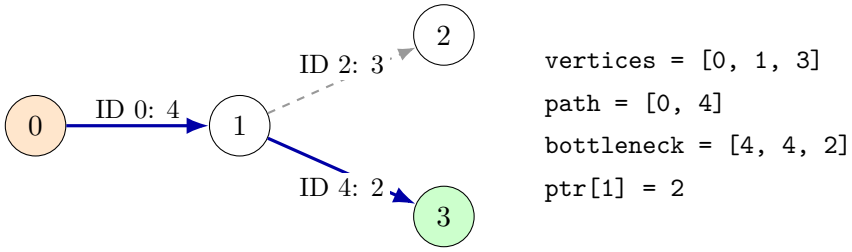
B. Remove the unsuccessful last step and resume at vertex 1.



ID 2 still has capacity 3. Its branch cannot continue in these levels, so skip it.

Position 2 of `adj[1]` contains ID 4. Follow it to the sink, append vertex 3 and edge ID 4, and append `min(4,2) = 2` to the bottleneck list:

C. The route 0 to 1 to 3 reaches the sink.

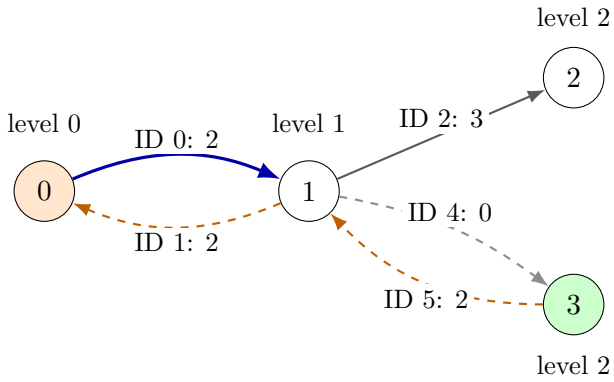


The last bottleneck is 2. Only now send 2 units along edge IDs 0 and 4.

At the sink, read the last bottleneck and update every edge ID in `path` together with its reverse. Here ID 0's capacity becomes `4 - 2 = 2`, and ID 4's becomes `2 - 2 = 0`; reverse IDs 1 and 5 each gain 2. The unused branch through vertex 2 still carries no flow.

3.3.4 After sending flow: a new search in the same phase

Continue with the same four-vertex graph. We have just sent 2 units on `0 -> 1 -> 3`. The drawing below shows the residual capacities afterwards:



Current total: 2. Labels are edge ID: remaining residual capacity.
ID 4 is now empty. Reverse IDs 1 and 5 have capacity, but go to smaller levels.
The zero-capacity reverse ID 3 is omitted.

Three kinds of state have different lifetimes:

- **Capacities and the accumulated total stay.** They describe the flow already sent. The total is 2.
- **Current pointers stay for this phase.** In particular, `ptr[0] = 0` and `ptr[1] = 2`. Vertex 1 has already discarded its first two outgoing entries.
- **The candidate route starts over.** A new search creates `vertices = [0]`, `path = []`, and `bottleneck = [4]`, using the same safe bound as before.

Keeping a pointer does not mean keeping the old route. It means remembering which outgoing entries have already failed under these fixed levels.

We could reset the pointers after every completed path and still obtain the correct answer. However, we would repeat searches already known to fail. Keeping the pointers avoids this repeated work.

The reason this is safe is that **an impasse stays an impasse within the same phase**. The levels are fixed, and the residual capacities of arrows advancing one level can only decrease. Sending flow creates cancellation permissions in the opposite direction, towards smaller levels in this phase; these new permissions are not allowed in this phase. Thus no new allowed continuation can appear behind an entry we have already discarded.

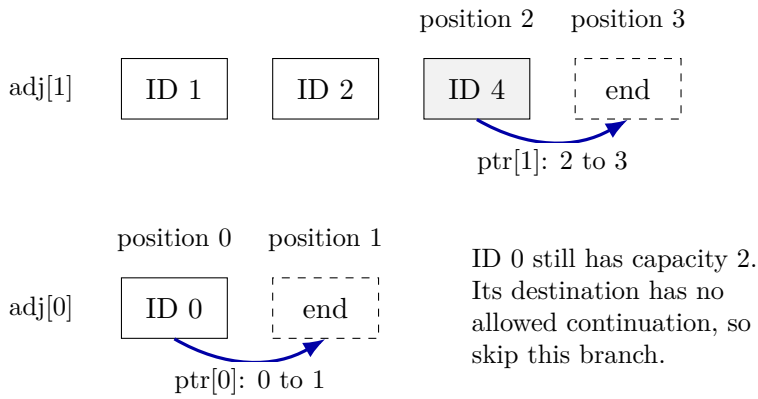
In our example, `adj[1] = [1,2,4]` and `ptr[1] = 2`. We already know that ID 1 goes to a smaller level and ID 2 leads to the dead end at vertex 2. Resetting `ptr[1]` to zero would merely check these two entries again.

Keeping a pointer does not skip an edge that is still useful. After the successful push, `ptr[0]` is still zero, selecting ID 0, which still has capacity 2. The next search will try it again, as we now follow step by step.

Follow the next search.

1. At vertex 0, position `ptr[0] = 0` selects ID 0. It still has capacity 2 and advances from level 0 to level 1, so follow it to vertex 1.

2. At vertex 1, resume at position `ptr[1] = 2`, selecting ID 4. Its capacity is now zero, so advance `ptr[1]` to 3. But `adj[1] = [1,2,4]` has only positions 0, 1, and 2: position 3 is past the end. No entry remains.
3. Return from vertex 1 to vertex 0, removing vertex 1 and ID 0 from the candidate route. Advance `ptr[0]` to 1. ID 0 still has capacity, but its destination has no allowed way onward, so this branch cannot reach the sink.
4. `adj[0] = [0]` has only position 0. The source's pointer is now past its end too. The candidate route empties and this search returns 0.



The dashed end boxes are past the lists, not stored edges.
ptr[1] = len(adj[1]) and ptr[0] = len(adj[0]) mean there is no entry left to try.

Returning 0 here means no additional flow was found. It does not erase the 2 units already sent. The inner loop ends because no route remains in these levels; Dinic then runs a new BFS on the updated residual graph.

On this particular graph, the new BFS reaches vertices 0, 1, and 2, but not sink 3. The algorithm therefore stops and returns the accumulated value 2. There is no further phase in this example.

If the new BFS does reach the sink on another instance, begin a new phase: keep the capacities and total, but reset all current pointers to zero. Entries rejected in the preceding phase may now belong to a new allowed route. The candidate-route lists also start over as usual.

Resetting at a new phase is therefore different from resetting after every path: new levels can change which directions are allowed. Pointer reuse within a phase saves work without ruling out a valid route; the efficiency discussion below explains how this gives the running-time bound. The explicit lists carry out the exploration and backtracking without recursive calls.

3.4 Why Dinic returns a maximum flow

The running-time bound alone does not prove that the answer is optimal. Correctness needs three facts: the current flow stays valid, a blocked phase cannot cause a premature return, and the final failed BFS certifies optimality.

3.4.1 Each augmentation preserves a valid flow and its reported value

The initial zero flow satisfies capacities and conservation. On a residual source-to-sink path, send its bottleneck `delta`. A forward residual step adds to its input edge's flow; a backward step subtracts from it. The bottleneck ensures that neither change crosses a capacity bound or makes an input flow negative.

At every internal path vertex, the two incident changes balance: the net change in incoming minus outgoing flow is zero, including when a step cancels an earlier assignment. The source supplies `delta` more in net flow, and the sink absorbs `delta` more. Thus the resulting assignment is feasible, and adding `delta` to the stored total gives its actual flow value.

3.4.2 Restricting one phase to its levels does not discard future routes

The restriction is temporary. Dinic retains the full residual graph; it only limits which arrows a path search may use during this phase.

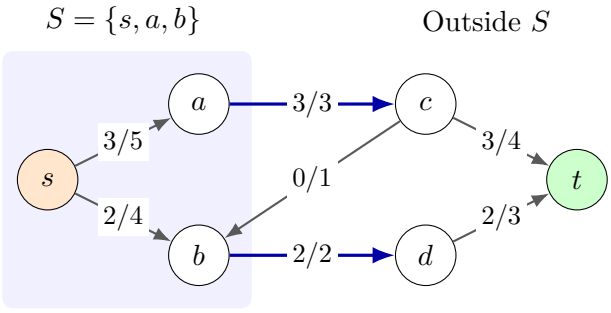
If BFS reaches the sink, a shortest residual route exists. Every arrow of that route advances one BFS level, so the new level graph contains a usable source-to-sink path. With current pointers reset, the search can find an augmentation; it is not prevented from starting by the level restriction.

After a blocking flow, the inner search returns zero. Dinic does **not** return the accumulated total yet: BFS again examines all positive residual arrows, without using the old current pointers to skip entries. It may find a longer route or one using a cancellation arrow. If so, reset the pointers and begin another phase. Only a failed BFS of the full residual graph ends the algorithm.

3.4.3 A failed full BFS proves that no larger feasible flow exists

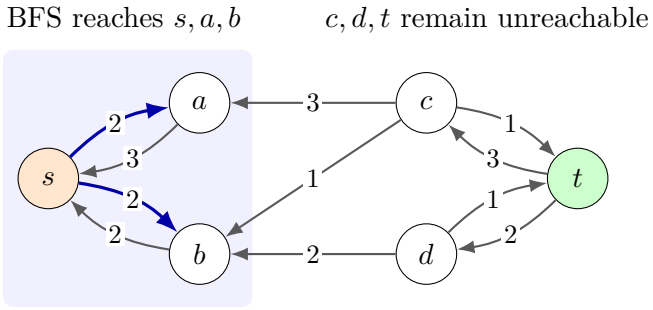
Run BFS from the source using **all positive residual arrows**, including cancellation arrows. Suppose it cannot reach the sink. Let `S` contain every vertex it did reach, not just the source. The sink lies outside `S`.

The following network has a feasible flow of 5. Labels show input flow/capacity. The shaded region will be the set reached by the final BFS:



Input edges: flow/capacity. Blue edges leave S.

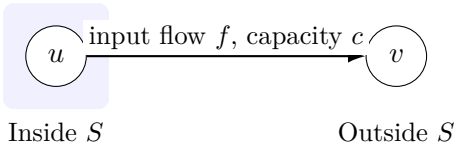
Now replace each input edge by its residual permissions. The next drawing shows **all positive residual arrows**; zero-capacity arrows are omitted:



Residual capacities. No positive arrow leaves the shaded region.

BFS reaches `a` and `b` from `s`, but has no residual arrow out of the shaded region. Arrows such as `c -> a` point **into** it, so they cannot help the search leave it. Why does this failure prove that 5 is optimal?

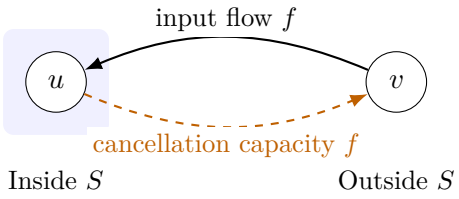
First, every input edge leaving `S` must be full. Here `a -> c` carries 3 out of 3, and `b -> d` carries 2 out of 2. If either had spare capacity, BFS could cross to its outside endpoint. In general:



Spare capacity $c - f > 0$ would let BFS leave S.

Since it cannot leave: $c - f = 0$, so $f = c$.

Second, every input edge entering `S` must carry zero flow. This also matters: flow coming back into `S` would reduce its net delivery. In our example, the entering input edge `c -> b` carries zero. If it carried a positive amount, its cancellation arrow `b -> c` would let BFS leave `S`:



If $f > 0$, the orange residual arrow would let BFS leave S.

Since it cannot leave: $f = 0$.

Finally, compare our delivery with the best any flow could deliver. Add conservation over all vertices of `S` except the source. Internal transfers cancel: a unit sent from `s` to `a`, for instance, stays within `S`. Only transfers across its boundary count towards the net delivery:

```
flow value = flow leaving S - flow entering S
```

In the example, the current value is `3 + 2 - 0 = 5`. No feasible flow can have a larger value: the only input edges leaving `S` have total capacity `3 + 2 = 5`. Reorganizing the flow inside either region cannot increase these capacities, and sending flow back into `S` can only reduce net delivery.

In general, let `C` be the sum of the input capacities leaving `S`. The two facts above give:

```
Our current flow:  value = C - 0 = C
Any feasible flow: value <= C
```

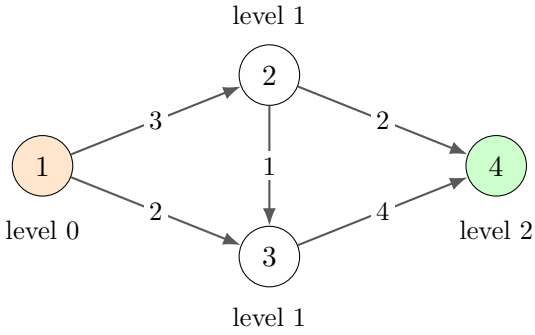
Our feasible flow attains an upper bound valid for every feasible flow, so it is maximum. This is why a failed BFS of the full residual graph is a certificate of optimality, rather than just a failed local search. The next subsection proves that Dinic reaches this condition in finitely many phases. Chapter 4 develops this boundary argument as a minimum cut.

3.5 Why Dinic finishes efficiently

There are two separate questions: how many phases can occur, and how much work one phase can do. Write `V` for the number of vertices and `E` for the number of input edges. The representation stores `2E` residual entries; that factor of two does not change an asymptotic bound.

3.5.1 Each successful phase makes the shortest remaining route longer

Consider this four-vertex network, starting with zero flow. The numbers on the arrows are input capacities. BFS assigns levels 0, 1, 1, and 2:



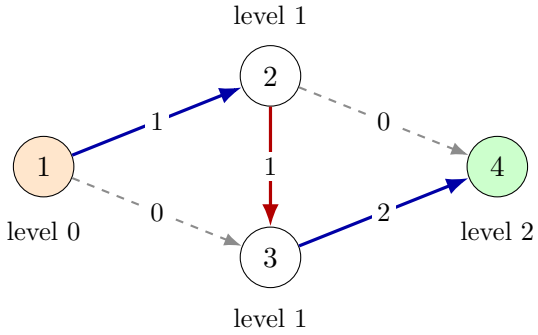
Start of phase 1.

Source: vertex 1.
Sink: vertex 4.
Flow value: 0.
Shortest distance: 2.

Input capacities before any flow is sent.

In phase 1, send 2 units along `1 -> 2 -> 4`, then 2 units along `1 -> 3 -> 4`. The total flow becomes 4. The arrows `2 -> 4` and `1 -> 3` now have zero forward residual capacity, blocking both two-arrow routes.

The drawing below shows the forward residual capacities after these two pushes, keeping **the same levels as above**. The route `1 -> 2 -> 3 -> 4` still has capacity, but its middle arrow stays at level 1, so the phase's flow search cannot use it:



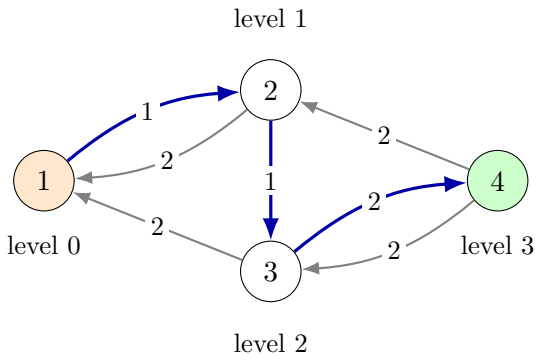
After the first phase: total 4.

At phase start: shortest routes have 2 arrows.
Blue: advances one level.
Red: same level, forbidden in this phase.

Forward residual capacities after phase 1. Positive reverse arrows are omitted here. The residual route has levels $0, 1, 1, 2$. Its red arrow prevents using it in this phase.

Route `1 -> 2 -> 3 -> 4` takes three steps. Using the levels computed for this phase, its level sequence is `0,1,1,2`: crossing `2 -> 3` uses a step without increasing the level. **The flow search does not take this route in phase 1.**

After phase 1 ends, run BFS again on the full residual graph. It assigns levels 0, 1, 2, and 3 to vertices 1, 2, 3, and 4 respectively:



All positive residual arrows, with the new BFS levels.
Residual capacities have not changed: BFS only recomputed the levels.

The middle arrow `2 -> 3` now goes from level 1 to level 2. The whole blue route `1 -> 2 -> 3 -> 4` advances one level at every step, so phase 2 can send its bottleneck of 1. The sink's shortest distance has increased from 2 to 3 between the starts of the two phases.

To understand why this always happens, keep the BFS numbers from the start of the phase written beside the vertices, even after capacities change. Call these fixed numbers `L[v]`. The source has `L[s] = 0` and the sink has `L[t] = D`. We will compare routes using these same numbers; we are not running a new BFS yet.

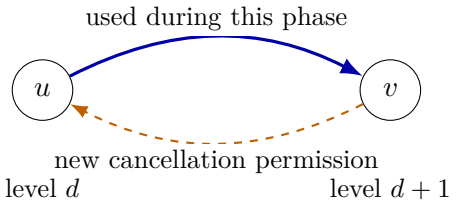
Before the phase, one residual arrow can increase `L` by at most 1. For example, suppose BFS assigns `L[u] = 2`. If a positive residual arrow `u -> v` exists, we can reach `v` in at most three steps: take a shortest two-step route to `u`, then that arrow. Thus BFS cannot assign `L[v] = 5`; it must assign `L[v] <= 3`. In general:

```
L[v] <= L[u] + 1
```

An arrow may go to the next level (`L[v] = L[u] + 1`), stay at the same level, or go to a smaller level. It cannot increase the level by more than 1.

This describes the **full residual graph**. The **flow search during this phase** follows only positive-capacity arrows satisfying `L[v] = L[u] + 1`. It cannot follow an arrow to the same level or to a smaller level, even if that arrow has positive residual capacity. The levels remain fixed throughout the phase; the next BFS will examine all positive residual arrows again.

Changing capacities during the phase preserves this limit. Arrows that remain usable already satisfied it. An arrow that becomes usable because we sent flow is the reverse of an arrow used by this phase. We only sent flow from a level `d` to level `d+1`, so the newly available reverse goes from `d+1` to `d`. It also cannot skip forward over a level:



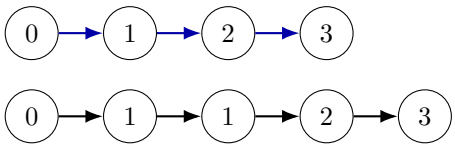
Sending flow forward makes its reverse available.

The reverse decreases the level by 1, so this phase cannot use it.

These are the levels computed by BFS for this phase. A later BFS may assign different ones.

Now ask whether a route of `D` arrows could remain after the phase. It would have to start at level 0 and finish at level `D`. Since each arrow increases the level by at most 1, it would have to increase it by exactly 1 on every step. There is no spare step for moving sideways or backward.

For instance, with `D = 3`, compare these sequences of fixed level numbers:



3 arrows: every step increases the level.

4 arrows: one step stays at level 1.

The circles show fixed BFS levels, not vertex IDs.

But the phase stopped precisely because **no positive-capacity source-to-sink route increasing the level at every step remained**. That is the definition of its blocking flow. So a route of `D` arrows cannot remain. A shorter one is also impossible: fewer than `D` increases of at most 1 cannot take us from 0 to `D`.

Consequently, any remaining route needs **more than `D` arrows**. Here is how this limits the number of phases.

Each new phase needs a larger sink distance. At the start of a phase, BFS finds the shortest residual source-to-sink route; let its length be `D`. After that phase, no route of length `D` or less remains. The next BFS therefore either fails, ending the algorithm, or finds a distance of at least `D+1`. In the latter case, we begin another phase. Thus the distances at the starts of successive phases are strictly increasing integers.

A finite shortest distance cannot exceed `V-1`. If a route visits a vertex twice, the part between those two visits is a cycle. Removing that part gives a shorter route with the same source and sink. So a shortest route never repeats a vertex: it visits at most `V` vertices and uses at most `V-1` arrows. This remains true when some arrows cancel earlier flow.

For example, suppose the graph has `V = 6` vertices and the first BFS finds the sink at distance 2. The distances at the starts of later phases can only come from this list, in order:

```
Possible phase-start distances:  2, 3, 4, 5
An execution might use:          2,    4, 5, then no route
```

Some distances may be skipped, but none can be repeated. Distance 6 cannot occur: a shortest route of six arrows would need seven distinct vertices, and this graph has only six. After a phase starting at distance 5, the next BFS must therefore fail.

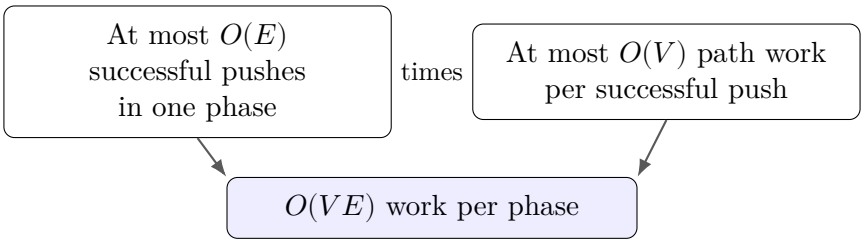
In general, the possible distances are the `V-1` integers from 1 to `V-1` (the source and sink are distinct). Each phase uses a different one, so there are at most `V-1` phases, followed by one final unsuccessful BFS. This gives `O(V)` phases. We are counting whole blocking-flow phases here; one phase may send flow along several routes.

3.5.2 Bound the work inside one phase

Failed entries are not retried indefinitely. Each `ptr[u]` only advances through `adj[u]`; it never moves backward during a phase. Across all vertices, there are `2E` entries to pass. Hence these advances and the dead branches they discard account for `O(E)` work.

Every successful push uses up at least one allowed edge. We send the path's minimum residual capacity, so some edge on that path becomes empty. It stays empty for this phase: restoring its capacity would require using its reverse, which decreases the level and is forbidden in this phase. There are only `2E` residual entries, so there are at most `O(E)` successful pushes.

One successful route has at most `V-1` arrows. Following and updating its path stacks costs `O(V)`. Several routes may share an edge that still has room; that repeated productive work is included in this `O(V)` per push. Together with the failed-entry work, one blocking phase costs `O(VE)`.



At most $O(V)$ phases
therefore total time $O(V^2E)$.

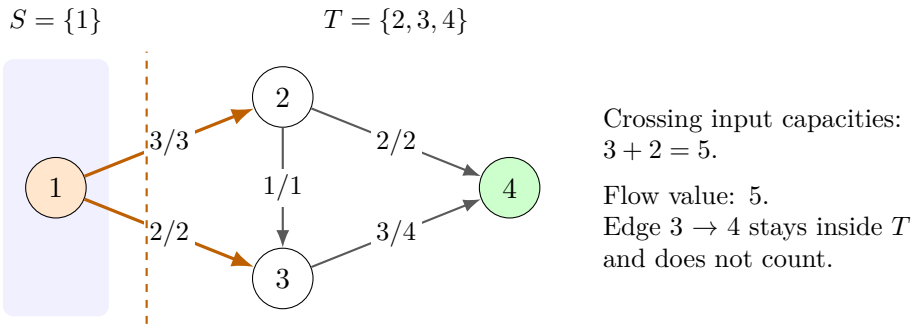
Multiplying `O(V)` phases by `O(VE)` work per phase gives Dinic's general `O(V^2*E)` time bound. BFS and pointer initialization cost `O(V+E)` per phase, which fit this bound for a nonempty graph. With no edges, one BFS immediately returns flow zero in `O(V)` time. The graph, levels, pointers, and path stacks use `O(V+E)` memory.

The bound explains both implementation choices: sending a full bottleneck avoids one operation per unit of capacity, and retaining current pointers avoids repeating failed searches within a phase.

4 Minimum cut: its capacity equals the maximum flow

A cut partitions vertices into `S` containing `s` and its complement `T` containing `t`. Its capacity is the sum of **input capacities** of edges pointing from `S` into `T`. Edges from `T` into `S` are not added.

Max-flow min-cut theorem. The largest feasible flow value equals the smallest cut capacity. We will prove both directions: every cut bounds every flow from above, and the final residual search finds a cut whose capacity equals the flow we have already sent.



Labels are input flow/capacity. The orange input edges leave the source side.

Conservation inside `S` cancels all internal flows. Hence the value of any feasible flow is

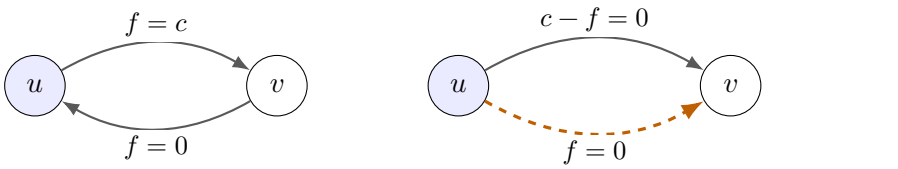
```
flow value = flow from S to T - flow from T to S
           <= capacity from S to T
```

Every cut is therefore an upper bound. After maximum flow, let `S` be all vertices reachable from `s` using positive residual edges. The sink is outside `S`, otherwise another augmentation would exist. Two facts now hold:

1. Every input edge from `S` to `T` is saturated. Otherwise its forward residual edge would make its destination reachable.
2. Every input edge from `T` to `S` carries zero flow. Otherwise its backward residual edge would make its other endpoint reachable.

The next figure uses two distinct input edges, one in each direction. On the right, both residual arrows point from `S` to `T`, but have different roles: the solid gray arrow would add flow to the outgoing input edge; the dashed orange arrow would cancel flow on the incoming input edge. Their labels show why both have zero capacity here.

Input flows across the cut Attempts to leave S in the residual graph



u in S v in T Solid gray: forward permission, capacity 0.
Dashed orange: cancellation permission, capacity 0.

Both zero-capacity residual attempts are unusable; neither makes v reachable.

Substitute these facts into the displayed equality: the flow value equals this cut's capacity. This proves the **max-flow min-cut theorem**:

Max-flow min-cut theorem

maximum flow value = minimum cut capacity

Why are both optimal? No feasible flow can exceed this cut's capacity, so our flow is maximum. No cut can have capacity below our flow's value, so this cut is minimum. The absence of incoming flow is needed as well as the saturation of outgoing edges to obtain the equality.

For the opening example, `S = {1}` and `T = {2,3,4}`. The crossing input capacities are `3` and `2`, totaling `5`. Edge `3 -> 4` still has unused capacity, but both its endpoints are in `T`; that does not affect the cut. Thus **maximum flow value = minimum cut capacity = 5**.

One BFS computes `S` in `O(V+E)` time. For its capacity, sum original capacities across the partition, rather than summing residual capacities (which are zero on the forward crossing edges). Minimum cuts need not be unique.

5 Dinic: complete reusable implementations

Both versions use 0-based vertices and nonnegative integer capacities. `add_edge(u, v, c)` returns the forward residual ID. `max_flow(s, t)` mutates the residual graph and returns the **additional** flow sent by that call; a second call without graph changes returns `0`. `reachable(s)` returns a Boolean array for the current residual graph. Call it after maximum flow to extract a minimum cut.

The C++ version uses `long long` for capacities and totals. Its caller must keep total capacities and answers within that type. The Python integers do not have this overflow limit. Neither version uses recursion.

5.1 C++

```cpp
#pragma once
#include <algorithm>
#include <cassert>
#include <queue>
#include <vector>

struct Dinic {
    struct Edge {
        int to;
        long long cap;
    };
    int n;
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj;
    std::vector<int> level, ptr;

    explicit Dinic(int n) : n(n), adj(n) {}

    int add_edge(int u, int v, long long capacity) {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(capacity >= 0);
        int edge = static_cast<int>(edges.size());
        adj[u].push_back(edge);
        edges.push_back({v, capacity});
        adj[v].push_back(edge ^ 1);
        edges.push_back({u, 0});
        return edge;
    }

    bool bfs(int s, int t) {
```

```
        level.assign(n, -1);
        level[s] = 0;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    queue.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    long long send(int s, int t, long long limit) {
        std::vector<int> vertices{s}, path;
        std::vector<long long> bottleneck{limit};
        while (!vertices.empty()) {
            int u = vertices.back();
            if (u == t) {
                long long delta = bottleneck.back();
                for (int edge : path) {
                    edges[edge].cap -= delta;
                    edges[edge ^ 1].cap += delta;
                }
                return delta;
            }
            while (ptr[u] < static_cast<int>(adj[u].size())) {
                int edge = adj[u][ptr[u]];
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && level[v] == level[u] + 1)
                    break;
                ++ptr[u];
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                vertices.pop_back();
                bottleneck.pop_back();
                if (!path.empty()) {
                    path.pop_back();
                    ++ptr[vertices.back()];
                }
            } else {
                int edge = adj[u][ptr[u]];
                path.push_back(edge);
                vertices.push_back(edges[edge].to);
                bottleneck.push_back(
                    std::min(bottleneck.back(), edges[edge].cap));
```

```
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        assert(0 <= s && s < n && 0 <= t && t < n && s != t);
        long long added = 0, limit = 0;
        for (int edge : adj[s]) limit += edges[edge].cap;
        while (bfs(s, t)) {
            ptr.assign(n, 0);
            while (long long delta = send(s, t, limit)) added += delta;
        }
        return added;
    }

    std::vector<bool> reachable(int s) const {
        std::vector<bool> seen(n, false);
        seen[s] = true;
        std::queue<int> queue;
        queue.push(s);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            for (int edge : adj[u]) {
                int v = edges[edge].to;
                if (edges[edge].cap > 0 && !seen[v]) {
                    seen[v] = true;
                    queue.push(v);
                }
            }
        }
        return seen;
    }
};
```

5.2 Python

```
from collections import deque


class Dinic:
    """0-based vertices; integer capacities; paired residual edge IDs."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]
        self.to = []
```

```
        self.cap = []

    def add_edge(self, u, v, capacity):
        assert 0 <= u < self.n and 0 <= v < self.n
        assert capacity >= 0
        edge = len(self.to)
        self.adj[u].append(edge)
        self.to.append(v)
        self.cap.append(capacity)
        self.adj[v].append(edge ^ 1)
        self.to.append(u)
        self.cap.append(0)
        return edge

    def _bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and self.level[v] == -1:
                    self.level[v] = self.level[u] + 1
                    queue.append(v)
        return self.level[t] != -1

    def _send(self, s, t, limit):
        vertices = [s]
        path = []
        bottleneck = [limit]
        while vertices:
            u = vertices[-1]
            if u == t:
                delta = bottleneck[-1]
                for edge in path:
                    self.cap[edge] -= delta
                    self.cap[edge ^ 1] += delta
                return delta
            while self.ptr[u] < len(self.adj[u]):
                edge = self.adj[u][self.ptr[u]]
                v = self.to[edge]
                if (self.cap[edge] > 0
                        and self.level[v] == self.level[u] + 1):
                    break
                self.ptr[u] += 1
            if self.ptr[u] == len(self.adj[u]):
                vertices.pop()
                bottleneck.pop()
                if path:
                    path.pop()
```

```
                    self.ptr[vertices[-1]] += 1
            else:
                edge = self.adj[u][self.ptr[u]]
                path.append(edge)
                vertices.append(self.to[edge])
                bottleneck.append(min(bottleneck[-1], self.cap[edge]))
        return 0

    def max_flow(self, s, t):
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        added = 0
        # A finite upper bound on every possible augmentation.
        limit = sum(self.cap[edge] for edge in self.adj[s])
        while self._bfs(s, t):
            self.ptr = [0] * self.n
            while True:
                delta = self._send(s, t, limit)
                if delta == 0:
                    break
                added += delta
        return added

    def reachable(self, s):
        seen = [False] * self.n
        seen[s] = True
        queue = deque([s])
        while queue:
            u = queue.popleft()
            for edge in self.adj[u]:
                v = self.to[edge]
                if self.cap[edge] > 0 and not seen[v]:
                    seen[v] = True
                    queue.append(v)
        return seen
```

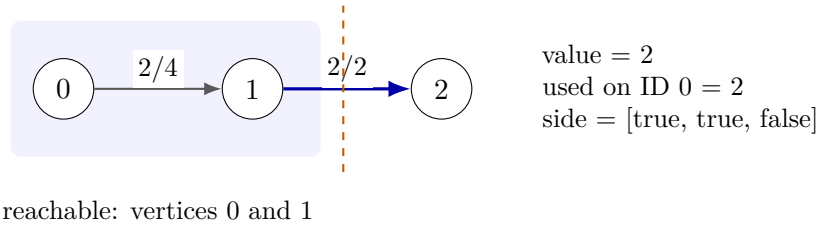
5.3 Small API example

This is a three-vertex network with a bottleneck of 2. It illustrates the structure's operations without an exercise input/output wrapper.

```
Dinic network(3);
int edge = network.add_edge(0, 1, 4);
network.add_edge(1, 2, 2);
long long value = network.max_flow(0, 2);  // 2
long long used = network.edges[edge ^ 1].cap;  // 2
std::vector<bool> side = network.reachable(0);  // true, true, false
```

```
network = Dinic(3)
edge = network.add_edge(0, 1, 4)
```

```
network.add_edge(1, 2, 2)
value = network.max_flow(0, 2)          # 2
used = network.cap[edge ^ 1]            # 2
side = network.reachable(0)            # [True, True, False]
```



reachable: vertices 0 and 1

Final input flow/capacity. The shaded vertices remain reachable through residual capacity.

6 Matching: a concrete reassignment

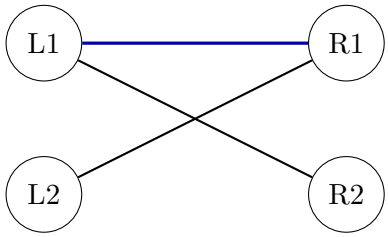
We now turn from routing amounts through a network to assigning individual resources. Suppose people request appointment slots. A person may accept several slots, but can receive at most one; each slot can serve at most one person. We want to accommodate as many people as possible. A locally valid assignment may prevent someone else from getting a slot, so we may need to reassign a person who was already accommodated.

Represent the possibilities by a **bipartite graph**: people are the left vertices, slots are the right vertices, and an edge means that this person accepts this slot. Every edge joins the two different sides. The edges here describe compatibility; their drawing direction has no meaning.

A **matching** is a selection of edges with no shared endpoint: no person or slot is used twice. Its size is the number of selected pairs. A **maximum matching** has the largest possible size; it need not match every vertex. A vertex is **free**, or **unmatched**, if no selected edge touches it.

The connection with flow is the idea of a reversible choice. We can remove an existing assignment as part of a larger rearrangement that adds one more pair overall. Let us see exactly how this works before describing the search algorithm.

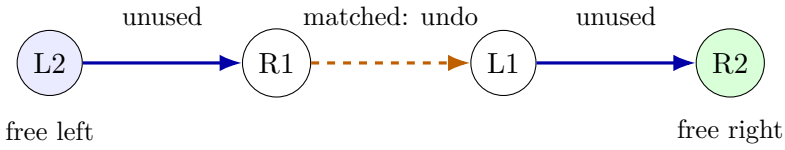
Take two people, L1 and L2, and two slots, R1 and R2. L1 accepts either slot; L2 accepts only R1. Initially, assign R1 to L1:



L1 can take R1 or R2.
L2 can take only R1.
Blue: first choose L1–R1.

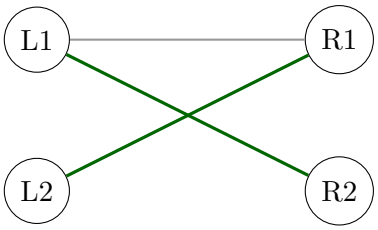
The initial choice `L1--R1` blocks L2 if we refuse to change it. R2 is still free, but L2 cannot take it directly. We can nevertheless accommodate L2 by moving L1 to R2. To find that rearrangement, search this route:

```
L2 -> R1 -> L1 -> R2
```



Follow unused edges left to right, and the occupied edge back to its owner.

Start at unmatched L2. Traverse an unused edge to R1, then its currently matched edge backward to L1, then an unused edge to unmatched R2. Flip the three edges: remove `L1--R1`, add `L2--R1` and `L1--R2`. The size rises from `1` to `2` and no vertex is used twice.



Remove L1–R1.

Add L1–R2 and L2–R1.

Both sides now use every vertex once.

Green edges form the new matching of size 2.

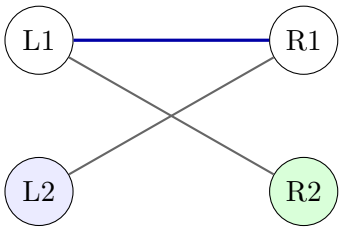
This is an **augmenting alternating path**. Its edges alternate between unused and matched, starting and ending with unused edges. It connects a free left vertex to a free right vertex and increases the matching size by one.

6.1 Search with owners and predecessors

The search answers a concrete question: **can we accommodate one more person by moving some current assignments?** It has two separate stages. First find a possible rearrangement without changing any assignment. If it reaches a free slot, apply the complete rearrangement.

6.1.1 What the arrays and queue remember

Continue with L1 assigned to R1 and L2 still free. Here is the complete starting state again:



Current partners:

left[L1] = R1; left[L2] = -1.
right[R1] = L1; right[R2] = -1.

Blue edge: assigned. Other edges: compatible.

`left[u]` is person `u`'s current slot. `right[v]` is slot `v`'s current owner. The value `-1` means no partner. The labels L1 and R1 name different vertices; in the implementation each side has its own 0-based IDs.

For one search, we also need:

- **`queue`: people whose possible slots still need inspection.** Initially it contains only the free person L2. We may later add an occupied slot's owner, to find out whether that person can move elsewhere.
- **`previous[v]`: the person from whom we first reached slot `v`.** It records a possible new owner, without assigning the slot yet. Initially every entry is `-1`, meaning the slot has not been visited by this search.
- **`seen[u]`: whether person `u` has already been enqueued.** This prevents repeated exploration of the same person's options.

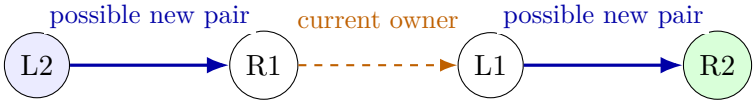
In particular, `right[R1]` and `previous[R1]` answer different questions: **who owns R1 now**, and **who might take R1 in this rearrangement**.

6.1.2 Follow the example without changing the matching

Start: `queue = [L2]`. All `previous` entries are `-1`.

Inspect L2: remove L2 from the queue. Its only possible slot is R1. Record `previous[R1] = L2`. But `right[R1] = L1`: R1 is occupied. We cannot finish yet, so enqueue L1 to look for somewhere its current owner could move. Now `queue = [L1]`. The assignment L1–R1 is still unchanged.

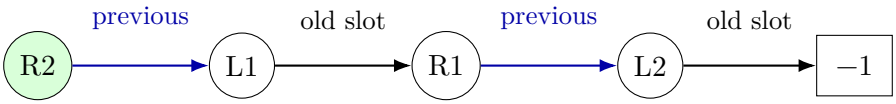
Inspect L1: remove L1 from the queue. R1 was already visited, so skip it. Its other possible slot is R2. Record `previous[R2] = L1`. Since `right[R2] = -1`, R2 is free: we have found an endpoint for the rearrangement.



L2 could take R1 if L1 moves. L1 can move to free R2.
Discovery is complete; no assignment has changed yet.

6.1.3 Apply the rearrangement from the free slot backward

Start with the free slot R2. `previous[R2] = L1` tells us to give it to L1. Before doing that, remember L1's old slot, R1: that is the next slot whose new owner we must set. Then `previous[R1] = L2` tells us to give R1 to L2. L2 had no old slot, so we are finished.



Give R2 to L1, then R1 to L2. Stop at the person who had no old slot.

These are exactly the reconstruction steps in the implementation:

```
v = the free slot found by the search
while v != -1:
    u = previous[v]       # person who will take this slot
    old = left[u]         # save that person's old slot first
    left[u] = v
    right[v] = u
    v = old               # continue with the slot just released
```

After the whole flip:

```
left[L1] = R2     right[R2] = L1
left[L2] = R1     right[R1] = L2
```

Treat the complete flip as one operation. Between its individual assignments, partner arrays may temporarily disagree; after it finishes they agree again. The matching has gained one pair while keeping every previous person matched.

6.1.4 Two more pairs, found in two visual passes

We will keep **the complete graph and the same vertex positions in every picture**. There are four people, A, B, C, D, and four slots. A can take R1, R3, or R4; B can take R2 or R3; C accepts only R1; D accepts only R2.

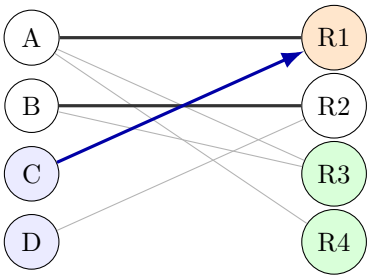
Process A, B, C, D in that order. First assign A to R1 and B to R2. We now have two pairs. The next two searches, from C and then D, will find the two missing pairs. Here a **pass** means one search and its resulting rearrangement; it is not a Dinic phase.

In each picture, gray edges show compatibility and **thick black edges show the current matching**. During a search, a **solid blue arrow proposes a new pair**; a **dashed orange arrow returns from an occupied slot to its current owner**. The orange vertex is the current search position. The matching changes only after reaching a free slot; the resulting pairs are then drawn in green.

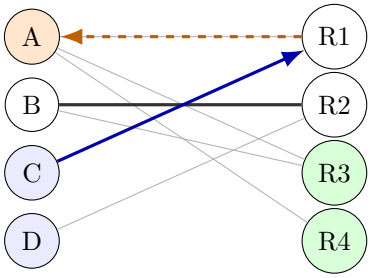
Pass 1: start from C, keeping A–R1 and B–R2.

C needs R1. Follow R1 back to its owner A, then find another slot for A. Read the pictures left to right, then top to bottom:

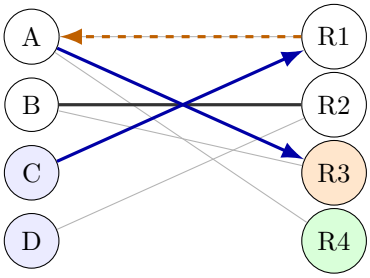
1. C wants R1, but R1 is occupied.



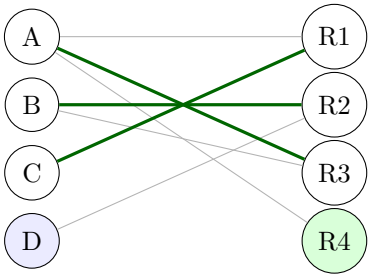
2. R1 belongs to A. Explore A again.



3. A can take free R3. The path is complete.



4. Apply the moves: A takes R3; C takes R1. Now 3 pairs.

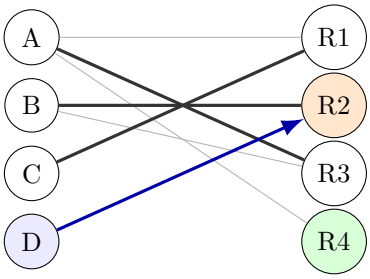


We removed A–R1 and added A–R3 and C–R1. B kept R2. Nobody lost their assignment, and C gained one. D remains free.

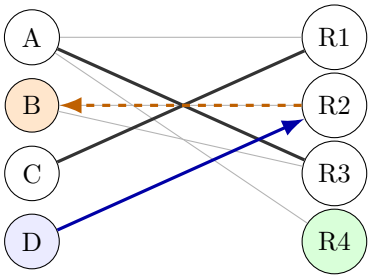
Pass 2: start from D, keeping the three pairs just obtained.

D needs R2, owned by B. B could move to R3, owned by A. So the search must return to **both B and A**, even though their turns as starting vertices are already over. A can move to free R4:

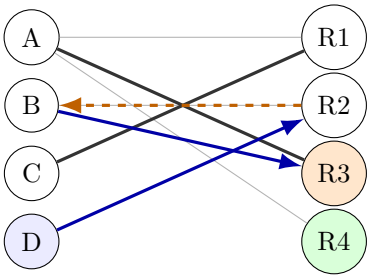
1. D wants R2, but R2 is occupied.



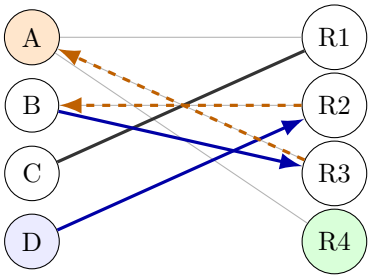
2. R2 belongs to B. Explore B again.



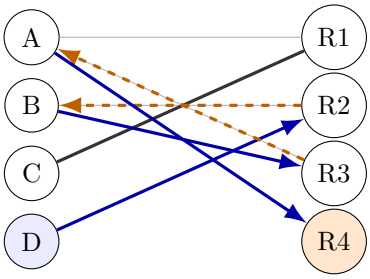
3. B could take R3, but R3 is occupied.



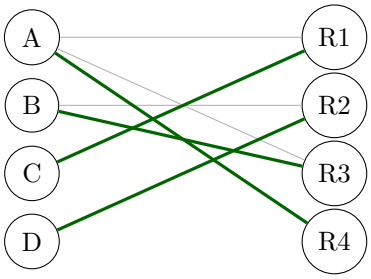
4. R3 belongs to A. Explore A again.



5. A can take free R4. The path is complete.



6. Apply: A takes R4, B takes R3, D takes R2. Now 4 pairs.

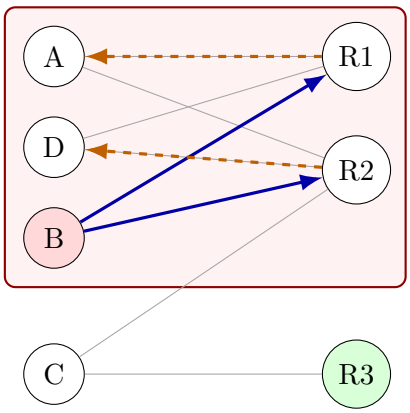


C kept R1. Removing two pairs and adding three increased the size from 3 to 4. This is optimal: all four people have different slots, and four people cannot form more than four pairs.

Why does one turn per starting person suffice? The searches may revisit earlier people to move their assignments, as we just saw with B and A. The real question is: **if someone's search fails, can a later rearrangement make it worth trying again?**

Consider this different graph, processed in the order A, D, B, C. A, D, and B each accept only R1 or R2. C accepts R2 or R3. First assign A–R1 and D–R2.

1. B fails after exploring a closed region. B tries both occupied slots, then all alternatives of their owners A and D:



B's search reaches A, D, R1, R2.

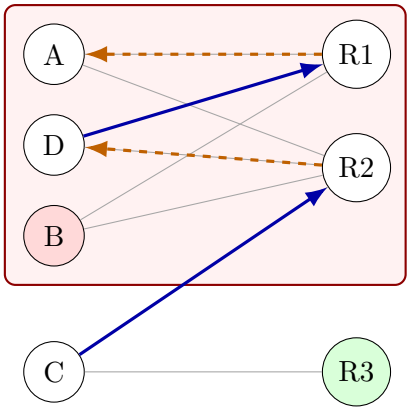
R1 and R2 are occupied.
A, D, B have no edge to R3.

No free slot can be reached.
B remains unmatched.

Shaded region: the vertices reached by B's failed search.
Black lines: assigned pairs; orange arrows follow assignments backward.

The failure establishes two facts about the shaded region: **every slot inside is occupied**, and **none of its people accepts a slot outside**. Changing the assignments elsewhere cannot create a missing compatibility edge to R3.

2. C may explore this region, but cannot succeed through it. Trying R2 leads to D, then possibly R1 and A. Once inside, the path cannot leave and cannot find a free slot:



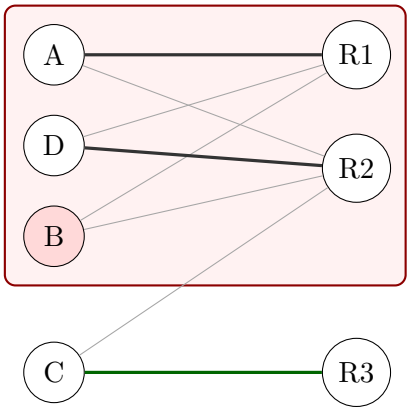
Try C–R2.

R2 leads to D, R1 leads to A, and A can only return to R1 or R2.

This branch cannot reach R3.
No assignment changes.

Shaded region: the vertices reached by B's failed search.
Black lines: assigned pairs; orange arrows follow assignments backward.

3. C succeeds outside, leaving B's region untouched. The successful path is C–R3. Only this winning path changes the matching:



C instead takes free R3.

A keeps R1.
D keeps R2.
B is still unmatched.

The closed region is unchanged.

Shaded region: the vertices reached by B's failed search.
Black lines: assigned pairs; orange arrows follow assignments backward.

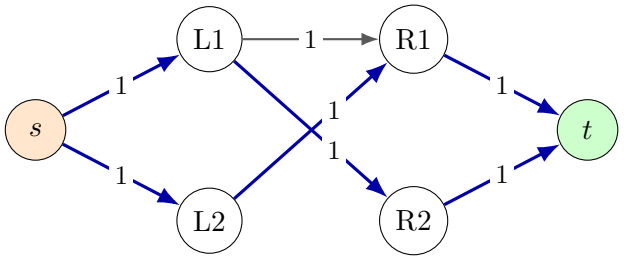
This is the general reason we do not retry failed roots. After an exhaustive failure, the reached region has no free slot and no alternating exit. Any later successful augmenting path must avoid it, so its assignments stay unchanged. Repeating the failed search would meet the same obstruction.

For a successful root, no retry is needed either: future augmentations may move that person to another slot, but never leave them unmatched. Thus we can start once from each person while still revisiting earlier matched people inside the searches.

6.2 Why matching is a flow problem too

Create a source, a sink, and distinct vertices for both sides. Add:

Edge family	Capacity
Source to every left vertex	1
Each allowed left-right pair	1
Every right vertex to sink	1



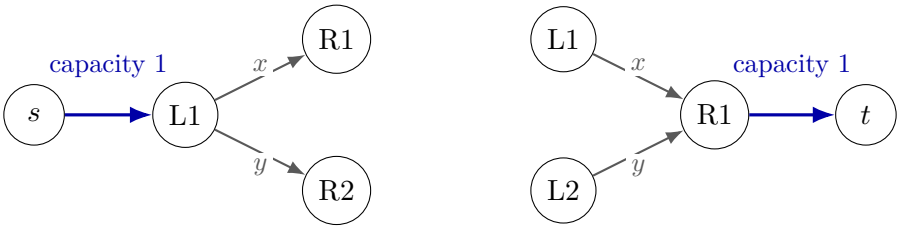
Every capacity is 1.

Blue routes send 2 units and select 2 pairs.

Source and sink edges prevent either side from using one vertex twice.

Why can a person receive at most one slot? Take L1. Its only incoming input edge is `s -> L1`, of capacity 1. Conservation at L1 says that the sum sent to all its possible slots equals the amount received from `s`. So L1 can send at most 1 in total. It cannot send 1 to R1 and another 1 to R2: that would require receiving 2 through an edge of capacity 1.

Why can a slot serve at most one person? Take R1. Its only outgoing input edge is `R1 -> t`, of capacity 1. Conservation at R1 says that the sum received from all people equals the amount sent to `t`. So R1 can receive at most 1 in total, even if several people accept it.



L1: $x + y \leq 1$.

R1: $x + y \leq 1$.

Two local views. Labels x, y are flows; blue labels are capacities.

Why do we get whole pairs rather than fractions? Dinic starts with zero flow. All capacities here are integers, so every residual capacity and every path bottleneck is an integer. Adding or cancelling that amount keeps all input flows integral. Since every middle edge has capacity 1, its flow is therefore either 0 or 1. We select exactly the pairs whose middle edge carries 1. The two constraints above ensure that no person or slot belongs to two selected pairs. These pairs form a matching, and their count equals the flow value.

How does a matching give a flow in the other direction? For each pair `L--R` in a matching, send one unit along `s -> L -> R -> t`. Because no person or slot is repeated, these routes never exceed any capacity. Conservation holds, and a matching of `k` pairs gives a feasible flow of value `k`. In the complete network at the start of this subsection, the two selected pairs give the routes `s -> L1 -> R2 -> t` and `s -> L2 -> R1 -> t`.

Thus any matching can be turned into a flow of the same value, and the maximum integral flow returned by Dinic can be turned into a matching of the same size. Neither optimum can exceed the other: **maximum matching size = maximum flow value in this constructed network**.

The matched step from R1 to L1 is exactly a reverse residual step. Our direct matching search leaves the source and sink implicit. It uses BFS augmenting paths without Dinic's blocking-flow phases; the two algorithms share the residual principle but organize their searches differently.

7 Matching: complete reusable implementations

`add_edge(u, v)` adds a permitted pair, with 0-based IDs on each side. `maximum_matching()` returns `(left, right)` pairs and leaves both partner arrays available. The implementation uses an explicit queue and path flip, so Python does not need recursive calls. Calling the method again without adding edges preserves the matching.

For `L` left vertices, `R` right vertices, and `E` allowed pairs, each search costs `O(E+L+R)`, including clearing its arrays. There are at most `L` searches, so total time is `O(L*(E+L+R))` and storage is `O(E+L+R)`. At the exercise's `L,R <= 500`, this direct approach is appropriate. For much larger bipartite graphs, Hopcroft-Karp batches shortest augmenting paths; it is a later tool.

7.1 C++

```cpp
#pragma once
#include <cassert>
#include <queue>
#include <utility>
#include <vector>

struct BipartiteMatching {
    std::vector<std::vector<int>> adj;
    std::vector<int> left, right;

    BipartiteMatching(int left_count, int right_count)
        : adj(left_count), left(left_count, -1), right(right_count, -1) {}

    void add_edge(int u, int v) {
        assert(0 <= u && u < static_cast<int>(left.size()));
        assert(0 <= v && v < static_cast<int>(right.size()));
        adj[u].push_back(v);
    }

    bool augment(int root) {
        std::vector<int> previous(right.size(), -1);
        std::vector<bool> seen(left.size(), false);
        std::queue<int> queue;
        seen[root] = true;
        queue.push(root);
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
```

```
            for (int v : adj[u]) {
                if (previous[v] != -1) continue;
                previous[v] = u;
                int owner = right[v];
                if (owner == -1) {
                    while (v != -1) {
                        u = previous[v];
                        int old = left[u];
                        left[u] = v;
                        right[v] = u;
                        v = old;
                    }
                    return true;
                }
                if (!seen[owner]) {
                    seen[owner] = true;
                    queue.push(owner);
                }
            }
        }
        return false;
    }

    std::vector<std::pair<int, int>> maximum_matching() {
        for (int root = 0; root < static_cast<int>(left.size()); ++root)
            if (left[root] == -1) augment(root);
        std::vector<std::pair<int, int>> pairs;
        for (int u = 0; u < static_cast<int>(left.size()); ++u)
            if (left[u] != -1) pairs.push_back({u, left[u]});
        return pairs;
    }
};
```

7.2 Python

```
from collections import deque


class BipartiteMatching:
    """Independent 0-based IDs on each side. -1 means unmatched."""

    def __init__(self, left_count, right_count):
        self.adj = [[] for _ in range(left_count)]
        self.left = [-1] * left_count
        self.right = [-1] * right_count

    def add_edge(self, u, v):
        assert 0 <= u < len(self.left) and 0 <= v < len(self.right)
```

```
        self.adj[u].append(v)

    def _augment(self, root):
        previous = [-1] * len(self.right)
        seen = [False] * len(self.left)
        seen[root] = True
        queue = deque([root])
        while queue:
            u = queue.popleft()
            for v in self.adj[u]:
                if previous[v] != -1:
                    continue
                previous[v] = u
                owner = self.right[v]
                if owner == -1:
                    # Follow predecessors and flip the entire path.
                    while v != -1:
                        u = previous[v]
                        old = self.left[u]
                        self.left[u] = v
                        self.right[v] = u
                        v = old
                    return True
                if not seen[owner]:
                    seen[owner] = True
                    queue.append(owner)
        return False

    def maximum_matching(self):
        for root in range(len(self.left)):
            if self.left[root] == -1:
                self._augment(root)
        return [(u, v) for u, v in enumerate(self.left) if v != -1]
```

7.3 Small API example

```
BipartiteMatching matching(2, 2);
matching.add_edge(0, 0);
matching.add_edge(0, 1);
matching.add_edge(1, 0);
auto pairs = matching.maximum_matching();  // {(0, 1), (1, 0)}
```

```
matching = BipartiteMatching(2, 2)
matching.add_edge(0, 0)
matching.add_edge(0, 1)
matching.add_edge(1, 0)
pairs = matching.maximum_matching()       # [(0, 1), (1, 0)]
```

8 Explore and test the algorithms

Open the standalone flow and matching page in a browser. It works offline. Choose a preset or enter your own edges, then step through BFS, path searches, residual updates, and matching flips. The highlighted code, variable values, and graph describe the same execution. Use the cancellation preset to watch the odd residual edge ID on the second augmentation. The page and the reusable implementations both use 0-based vertex IDs, including in the custom graph input.

8.1 Recognize a suitable model

Capacity restrictions, conserved quantities passing through junctions, and maximizing total delivery suggest flow. Choosing distinct pairs across two sides suggests matching. In both settings, a locally convenient choice can block later choices, and residual searches allow reassignment.

A capacity limits an amount; it is not a cost or a distance. Ordinary maximum flow does not minimize travel cost. Before implementing, identify the source, the sink, the capacity of each edge, and the quantity one unit represents. For matching, identify both sides and which pairs are legal.

8.2 Useful checks and failure cases

Case	Property to check
Single edge of capacity 7	Value is 7, not 1
Unreachable sink, or zero-capacity edges	Value is 0
Four-vertex opening network	Rebuild BFS; total is 5
Six-vertex cancellation network	Reverse residuals allow total 2
Parallel and opposite input edges	Every input edge keeps its own pair
Many capacities of a billion	C++ totals use 64-bit integers
A long directed chain	Explicit stack works without recursion
Two left vertices competing for one right	Matching size is 1
L1 connects to R1/R2, L2 only to R1	Reassignment finds size 2
No matching edges	Empty pair list

For small flow graphs, enumerate every source-side subset containing `s` but not `t`, compute its input cut capacity, and compare the minimum with Dinic. Also verify capacities, junction conservation, and the returned cut. For matching, use an exhaustive small search and validate every returned pair, its membership in the input, and endpoint uniqueness.

Before submitting, check both residual updates, pointer resets per phase, positive-residual BFS, independent matching IDs, and output ordering required by the statement. Public `solve.*` files are stubs; keep personal attempts in the private submissions overlay.

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
python3 sections/45_flows_and_matchings/implementations/check.py
node sections/45_flows_and_matchings/flow_playground/tests/engine.test.mjs
```

9 Exercises

Attempt A-C after studying the corresponding operations. Each statement specifies its input numbering and output format. For D, read its README and make an independent attempt before opening the editorial.

- A. Max Flow: Compute maximum delivery from vertex `1` to vertex `n`.
- B. Bipartite Matching: Find the maximum number of disjoint permitted pairs.
- C. Min Cut Reachable Side: Return the maximum value and final residual source side.
- D. Project Selection: Blind practice; use the standalone problem statement.